UNIVERSITY OF WOLLONGONG IN DUBAI

Assignment Cover Sheet

Subject Code:	CSCI316
Subject Name:	Big Data Mining Techniques and Implementation
Submission Type:	Project Report
Assignment Title:	Project 2
Student Name:	Basil Yusuf Habib, Sivajith Ajith Kumar, Suwathi Rajasekar, Adalia Alex, Asil Habib, Samyukth Kalathil, Masuma Calcuttawala, Disha Gurav, Mishaim Jat, Ashfina Abbas
Student Number:	8392237,8404744,8393606,8373759,8392201,8444109,8288185, 8331583,8792501,8420427
Student Email:	byh682@uowmail.edu.au, sak481@uowmail.edu.au, srr319@uowmail.edu.au, asa795@uowmail.edu.au, ayh157@uowmail.edu.ya, spk886@uowmail.edu.au, mmc951@uowmail.edu.au, dg268@uowmail.edu.au, Maj884@uowmail.edu.au, aa8433@uowmail.edu.ae
Lecturer Name:	Dr Patrick Mukala
Due Date:	20th March 2026
Date Submitted:	20th March 2026

PLAGIARISM:	**DECLARATION:**
The penalty for deliberate plagiarism is FAILURE in the subject. Plagiarism is cheating by using the written ideas or submitted work of someone else. UOWD has a strong policy against plagiarism. The University of Wollongong in Dubai also endorses a policy of non-discriminatory language practice and presentation. **PLEASE NOTE:** STUDENTS MUST RETAIN A COPY OF ANY WORK SUBMITTED	I/We certify that this is entirely my/our own work, except where I/we have given fully-documented references to the work of others, and that the material contained in this document has not previously been submitted for assessment in any formal course of study. I/we understand the definition and consequences of plagiarism. **Signature of Student:**

Optional Marks:

Comments:

Lecturer Assignment Receipt (To be filled in by student and retained by Lecturer upon return of assignment)

Subject: CSCI316
Assignment Title: Project Report
Student Name: Basil Yusuf Habib, Sivajith Ajith Kumar, Suwathi Rajasekar, Adalia Alex, Asil Habib, Samyukth Kalathil, Masuma Calcuttawala, Disha Gurav, Mishaim Jat, Ashfina Abbas
Student Number: 8392237,8404744,8393606,8373759,8392201,8444109,8288185, 8331583,8792501,8420427
Due Date: 20th March 2026
Date Submitted: 20th March 2026
Signature of Student:

Student Assignment Receipt (To be filled in and retained by Student upon submission of assignment)

Subject: CSCI316
Assignment Title: Project Report
Student Name: Basil Yusuf Habib, Sivajith Ajith Kumar, Suwathi Rajasekar, Adalia Alex, Asil Habib, Samyukth Kalathil, Masuma Calcuttawala, Disha Gurav, Mishaim Jat, Ashfina Abbas
Student Number: 8392237,8404744,8393606,8373759,8392201,8444109,8288185, 8331583,8792501,8420427
Due Date: 20th March 2026
Date Submitted: 20th March 2026
Signature of Lecturer

CSCI316 – Big Data Mining and Applications

Gulf Arabic – English Code-Switched Sentiment Analysis with Transfer Learning

Table of Contents

Introduction 4
Task and Language Justification 5
Why **Gulf Arabic?** 5
Why **Sentiment Analysis?** 5
Why **Transfer Learning?** 6
Dataset Description 6
Data Sources 6
Data Composition 7
Data Size 8
Dataset Preprocessing 8
Data Loading & Processing 8
Arabic Text Normalization 8
Gulf Dialect Normalization 9
Emoji Handling 9
Label Encoding 9
Dataset Splitting 9
Tokenization 10
Methodology 10
MARBERTv2 Model 10
Transfer Learning Strategies 10
Full Fine-Tuning 10
Parameter-Efficient Fine-Tuning (LoRA) 11
Mathematical Formulation 11
Key Implementation Details 11
Layer Injection Strategy 11
Parameter Efficiency 12
LoRA performs effectively in this context 12
Classification Head 12
Pipeline Implementation 12
Hyperparameter Configuration for both pipelines 13
Evaluation Metrics 14
Experiments and Results 14
Training Dynamics 14
Pytorch Training Results: 15
HuggingFace Training Results: 16

Cross-Framework Comparison 16
Cultural Evaluation - Code-Switch Sensitivity Score (CSS) 16
Motivation and Design 16
Computation Method 17
Justification 18
Human Evaluation 18
Deployment 18
Discussion 20
What Transferred Successfully 20
Anticipated Failure Modes 21
Full Fine-Tuning vs LoRA 21
LoRA (Parameter-Efficient Fine-Tuning) 22
Conclusion 22
Future Work 22
References 23
Appendix 23
Project Resources 23

Introduction

Natural Language Processing has moved fast thanks to big language systems like BERT and GPT - models built on transformer designs. Because of them, jobs like sorting text, translating speech, or spotting emotions in words work much better now. Still, not every tongue gets the same boost globally when it comes to these tools. English leads the pack since most top-tier models learn mainly from it, while countless other tongues and regional ways of speaking get left behind.

You see it clearly when looking at Arabic dialects. Even if experts look at Modern Standard Arabic now and then, dialects like Gulf Arabic stay mostly ignored. In places including the UAE, Saudi Arabia, Kuwait, Bahrain, because of daily use by so many people, it matters deeply there. Still, hardly any software or collections of spoken examples handle this form properly. Millions use it, but machines still struggle to keep up.

Now imagine a place where voices blend like city traffic UAE fits that sound. Conversations online often jump from one tongue to another mid-sentence, no pause. Sometimes it's Gulf Arabic starting strong, then slipping into English by the end. This switch isn’t rare it shows up daily across platforms. One moment you're reading colloquial phrases, next they’re tied with English words. Language there doesn’t stay in lanes. Sentences grow wilder when cultures meet fast.

For example:

"وما كان زين late الطلب وصل وايد"

Take this case: an Arabic sentence using the English term *late*. Most standard sentiment systems stumble here, built as they are for one language at a time. When forced into separate boxes English versus Arabic the tools break down. Monolingual setups miss context; Arabic-focused ones trip over regional speech patterns.

Consequently, those working in business, governance, or research within the UAE struggle to interpret public sentiment found in social media posts mixing languages tools suited for such content remain inadequate.

This study introduces a method aimed at analyzing sentiment within Gulf Arabic-English social media posts that switch between languages. Built on strong multilingual foundation models, it adjusts their behavior through targeted training steps applied to carefully selected mixed-language examples. While originally trained on diverse linguistic inputs, the system refines its understanding by focusing on real-world instances where speakers blend both tongues naturally.

This work shows accurate sentiment detection is possible with limited language data. Using knowledge from prior models instead of building new cuts resource demands significantly. Comparisons across different transfer learning strategies reveal which performs best under constraints. Results suggest efficiency improves when leveraging pre-trained systems carefully. Performance remains strong even where training material is sparse.

Task and Language Justification

Why **Gulf Arabic?**

Arabic is what most people speak daily in the UAE and nearby areas. Spoken often, yet rarely shows up in natural language processing studies when set beside Modern Standard Arabic. Casual chats online tend to flow in this form of speech. The choice makes sense because real conversations happen here, not just formal texts. Even though it's common, researchers still overlook it quite a bit. What stands out is how much presence it has socially, despite limited academic attention. This gap matters more than many realize. Focus lands on Gulf Arabic simply because that's where voices actually live. Not chosen at random, rooted in actual use across homes and digital spaces. Its absence in tech tools feels louder given how widely it spreads naturally.

Gulf Arabic sounds worlds apart from Modern Standard Arabic when you listen closely, word choices shift, rules bend, even the rhythm changes. Because of that twist, computers taught only in textbook Arabic often trip up trying to make sense of everyday talk in the Gulf region.

Out here, people in the UAE often blend Arabic and English when they speak, simply because of how diverse the society is. This mix slips through the cracks of most regular NLP data collections. Language ends up shifting mid-sentence, something these systems aren't built to catch.

What sets Gulf Arabic apart is how little data exists for it, turning it into a real test case for transfer learning methods. Though widely spoken, its digital footprint stays thin, which pushes researchers to try new ways of adapting models across languages.

Why **Sentiment Analysis?**

Focusing on emotions in words made sentiment analysis a natural pick, it shows up everywhere, from labs to real-world systems. Sorting phrases by their mood, say upbeat, downbeat, or flat, lies at its core.

Looking at feelings in text brings real benefits in the UAE. Because emotions show up clearly, companies track what customers really think of offerings. Public reactions to rules and city services become clearer when officials take a closer look. When scholars explore how minds shift over time, they lean on emotion tracking too.

On top of that, measuring feelings in text gives solid numbers like how often it's right or the F1 value, so testing things step by step works well.

Why **Transfer Learning?**

Starting fresh with a deep learning language system means gathering massive amounts of data along with serious computing power. These days, most transformer setups learn by processing anywhere from hundreds of millions to several billion bits of written text.

Yet not enough data exists for Gulf Arabic sentiment tasks; most sets have just thousands of entries. Starting fresh with so little material leads to weak results.

Starting off fresh isn't always needed. When one model moves from big multilingual data to a focused job, it uses past knowledge. What it already knows about language stays useful. Small sets of labelled examples guide its shift toward new goals.

Starting here, borrowed smarts from past models help it catch how Gulf Arabic mixes with other languages. That shift shapes its grasp without starting fresh.

Dataset Description

A major hurdle here involves missing a full collection of Gulf Arabic–English mixing examples tagged by emotion. Because of that gap, gathering data from several origins became the chosen path.

213,255 samples form the complete collection, pulled from various origins like pure Arabic, blended languages, plus synthetic inputs shaped by breadth and variety. Following cleanup and equalization steps, 84,728 entries went into model learning, while what was left got split between checking progress and final checks. Native Arabic passages appear here alongside switched language segments, together with made up instances built to cover missing pieces and mirror how people actually speak.

The dataset includes three sentiment classes:

- Positive
- Neutral
- Negative

Data Sources

Source	Type	Description
ASTD (Arabic Sentiment Twitter Dataset)	Arabic	Manually labeled Twitter sentiment data
ArbML Twitter Corpus	Arabic	High-quality Arabic sentiment tweets
Company Reviews (Kaggle)	Mixed	Reviews mapped from star ratings
ArE-CSTD	Code-switched	Synthetic Arabic-English code-switching dataset

MagedSaeed Dataset	Code-switched	Real-world mixed-language text
App Store Reviews	Mixed	User reviews with rating based labels
YouTube Comments	Mixed	Scraped UAE specific comments

This multi-source approach ensures that the dataset captures both formal Arabic and informal social media language, including real-world code-switching patterns.

Data Composition

Metric	Value
Total Samples	213,255
Training Set	149,957 (70.3%)
Validation Set	29,105 (13.6%)
Test Set	31,743 (14.9%)
Code-Switched Data	76.5%
Pure Arabic Data	23.5%

Class Distribution:

Class	Percentage
Positive	38.5%
Neutral	36.2%
Negative	25.3%

Data Size

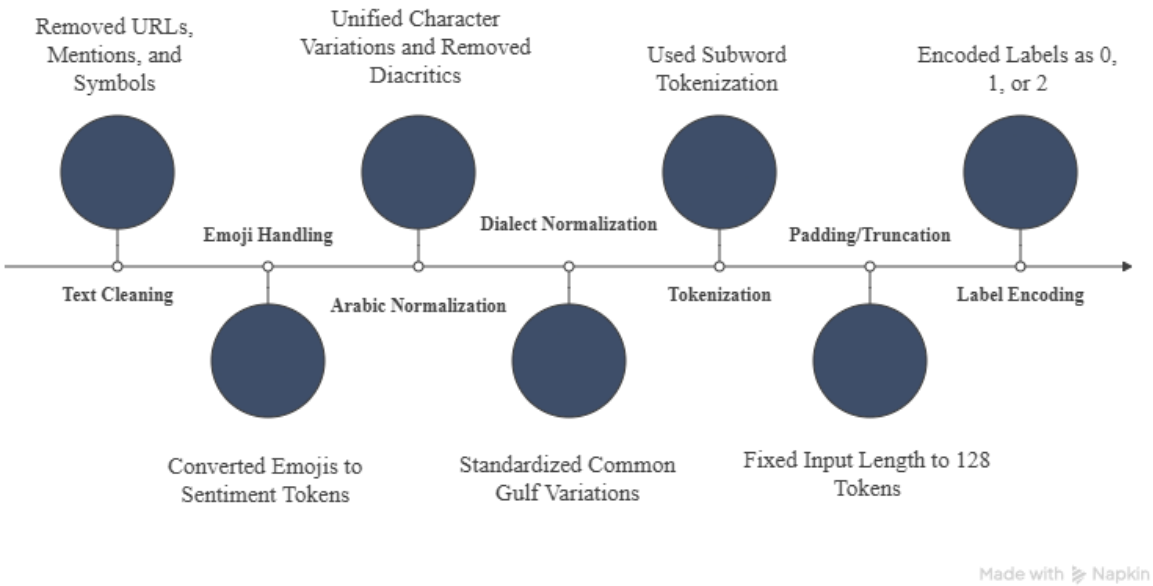
The final dataset contains approximately:

- Training set: 149,957 samples
- Validation set: 29,105 samples
- Test set: 31,743 samples

Each sample is labeled with one of three sentiment classes:

- Positive
- Negative
- Neutral

Dataset Preprocessing



Data Loading & Processing

- The dataset is loaded using pandas.read_csv().
- There is no special handling for encoding issues or combining multiple datasets.
- The code is flexible and can work with any properly formatted CSV file.

Arabic Text Normalization

Arabic text can have many different forms of the same letters, which can confuse the model.

To fix this, we normalize the text by:

- Converting different Alef forms (ا → آ ,إ ,أ)
- Converting (ي → ى)
- Converting (ه → ة)
- Removing diacritics (تشكيل)
- Removing elongation (-)

This helps make the text more consistent and easier for the model to understand.

Gulf Dialect Normalization

Gulf Arabic often includes informal words and repeated characters.

The code handles this by:

- Standardizing common dialect words and variations
- Reducing repeated characters (e.g., هه → هههههه)
- Normalizing common words like وايد

This makes the model better at handling real-life UAE-style text.

Emoji Handling

Emojis are important because they show sentiment.

Instead of removing them, we:

- Convert them into tokens like [POS_EMOJI], [NEG_EMOJI], [NEU_EMOJI]
- Remove emojis that are not recognized

This helps keep the emotional meaning in the text.

Label Encoding

The labels are converted from text to numbers:

0 = Negative

1 = Neutral

2 = Positive

This means the model does 3-class classification, not binary.

Dataset Splitting

The data is split into:

- 80% training
- 10% validation
- 10% testing

The split is done in order (not stratified), so class balance is not guaranteed.

Tokenization

A HuggingFace tokenizer is used

The text is converted into:

- input_ids
- attention_mask

Maximum length is set to 128 tokens.

Methodology

The project methodology uses transfer learning to implement pre-trained transformer-based encoder models for sentiment analysis on Gulf Arabic English code switched text.

MARBERTv2 Model

MARBERTv2 is an Arabic focused transformer model which was trained on 1 billion Arabic tweets that contain both dialectal and Gulf Arabic language data. The system contains 163 million parameters which enable it to function as an optimal choice for processing informal Arabic text and dialects.

Transfer Learning Strategies

Full Fine-Tuning

Every part of the pre-trained model gets adjusted when trained fully on the new data.

Some 110 million adjustable elements shape MARBERT runs with around 163 million tuned parts instead

Starting at 0.00002, the learning pace shifts slowly upward before tapering off toward the end. Step by step, it adjusts rising first, then fading as training progresses

Risk: catastrophic forgetting of pre-trained knowledge

Slowing down training helps. Pausing before things go off track makes a difference. Cutting gradients short stops wild updates. Each step plays its part without flash or promise

Fewer shortcuts here mean the system fits the data better, though it needs more processing power to pull off.

Parameter-Efficient Fine-Tuning (LoRA)

LoRA tweaks big transformer models without touching every weight. The method sidesteps full updates to the main weight matrix W. It works by adding lightweight adjustments on the side. Think of small changes that still make a difference. Original performance stays intact. Training becomes faster. Memory use drops. No need to retrain everything from scratch. Efficiency jumps when scaling across tasks. Updates live outside the core model. Switching adaptations takes less effort. Pretrained knowledge remains untouched. Changes are modular. Plugging in new tasks feels lighter. Heavy lifting is avoided. Flexibility increases quietly. Adjustments stack neatly when needed. Core architecture does not budge

A twist in the design: LoRA uses compact layers to handle new tasks without altering the original model's core. What shifts is how adjustments are shaped - slim matrices do the work, leaving everything else untouched.

A hands-on approach brought LoRA to life directly in PyTorch, built step by step from scratch. This way, every piece of math and structure became clear through doing, not just theory. With nothing added beyond core tools, the inner workings revealed themselves naturally during development.

Mathematical Formulation

For a weight matrix $W \in R^{d \times k}$, LoRA models the update as:

$\Delta W = \alpha/r \cdot B \cdot A$ Where:

- $A \in R^{r \times k}$
- $B \in R^{d \times r}$
- r = rank (set to 8)
- α = scaling factor (set to 16)

The forward pass becomes: $h = Wx + \alpha/r\ (BAx)$

Key Implementation Details

- The original weights W remain fully unchangeable throughout the entire process.
- Only matrices A and B are trainable
- Matrix B is initialized to zeros which establishes that:
- The initial output will match the pretrained model
- The system will maintain its original performance level from the beginning of training.

Layer Injection Strategy

Inside the setup, LoRA modules got placed at:

- Query projection layers
- Value projection layers

Here is why these layers matter so much:

- Attention weights fall under their command
- They strongly influence contextual understanding

Parameter Efficiency

Example per layer:

- Full Fine-Tuning:
  768×768=589,824 parameters

- LoRA:
  (768×8)+(8×768)=12,288 ~48× reduction per layer

Overall:

- Full model: ~163M trainable parameters
- LoRA: 297,219 (~0.18%)

LoRA performs effectively in this context

The task adaptation (general Arabic → sentiment classification) is:

- Low complexity relative to pretraining
- Fits inside a space with fewer dimensions

Classification Head

Classification architecture:

- Input: [CLS] token representation
- Dropout: 0.3 (for regularization)
- Linear layer: 768 → 3 output classes (negative, neutral, positive)
- Loss function: CrossEntropyLoss

Pipeline Implementation

We implemented the same training pipeline in two different frameworks to show we understand both low-level PyTorch and the HuggingFace ecosystem.

The first pipeline uses raw PyTorch. Everything here is written manually - the training loop, the optimizer, the learning rate scheduler, gradient clipping, early stopping, and evaluation. For LoRA specifically, we built the entire implementation from scratch in a separate file called peft_implementation.py without using any external library. This file defines the LoRALinear class and the inject_lora() function that replaces the query and value projection layers in MARBERTv2. The idea behind doing it from scratch was to actually understand the maths rather than just calling a library function. Data is fed in as PyTorch DataLoaders where each batch is a dictionary of input_ids, attention_mask, and label tensors.

```
PS C:\Users\Sivajith\Desktop\CSCI316 Project\Project 2> & C:/Users/Sivajith/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/Sivajith/Desktop/CSCI316 Pr
oject/Project 2/peft_implementation.py"

Testing LoRA implementation...

Before LoRA: 12,480 trainable parameters
LoRA applied: 2 layers replaced
Trainable parameters: 1,024 / 13,504 (7.58%)
After LoRA:  1,024 trainable parameters

Forward pass output shape: torch.Size([2, 10, 64])  (expected torch.Size([2, 10, 64]))
Parameters with gradients: ['query.lora_a', 'query.lora_b', 'value.lora_a', 'value.lora_b']
```

running peft_implementation.py

The second pipeline uses the HuggingFace Trainer API. Here, instead of writing out the training loop manually, we configure a TrainingArguments object and call trainer.train(). The Trainer handles everything internally - optimizer, scheduler, evaluation, checkpointing, and early stopping through a callback. For LoRA in this notebook, we used the HuggingFace peft library with LoraConfig instead of our scratch implementation. We made this choice because the Trainer is designed to work with peft's PeftModel objects natively. Both LoRA implementations use the same configuration — rank 8, alpha 16, targeting the query and value layers — and their results came within 0.13% of each other, which confirms both are doing the same thing correctly.

At the end of HuggingFace pipeline, we load the PyTorch results CSV from Pytorch pipeline and combine them with the HuggingFace results to produce the cross-framework comparison chart.

Hyperparameter Configuration for both pipelines

Hyperparameter	Value
Batch size	16
Max sequence length	128 tokens
Max epochs	3 with early stopping
Early stopping patience	3
LR — Full FT	2e-5
LR — LoRA	3e-4
Weight decay	0.01
Warmup ratio	0.1
Gradient clipping	max_norm = 1.0
LoRA rank	8
LoRA alpha	16.0
LoRA target layers	query, value
Optimizer	AdamW
Primary metric	Macro-averaged F1
Seed	42

Evaluation Metrics

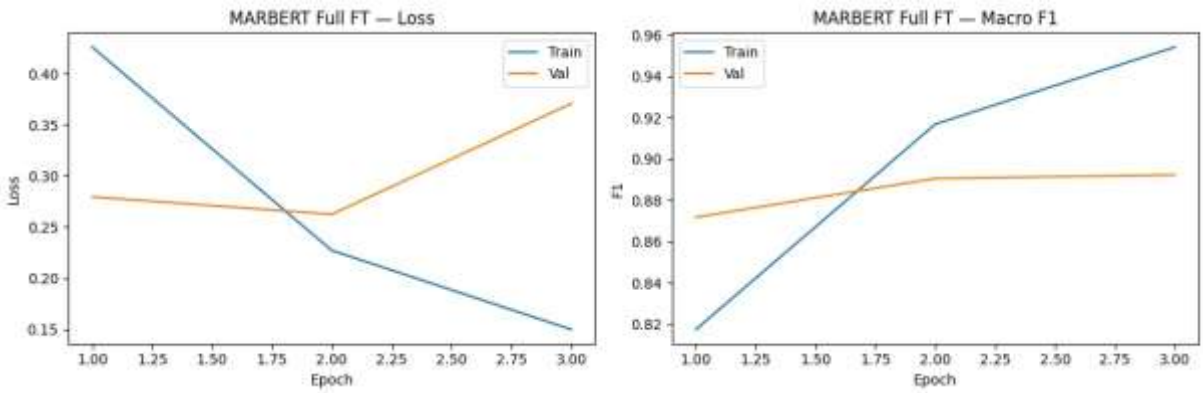
The primary evaluation metric is Macro-averaged F1 score, computed as the unweighted mean of per-class F1 scores across the three sentiment classes. This was chosen over accuracy because the test set retains its natural class distribution (approximately 75% neutral, 14% positive, 11% negative after split assignment). A trivial classifier predicting 'neutral' for every input would achieve approximately 75% accuracy but only 28.5% Macro-F1. Macro-F1 thus provides an honest measure of per-class performance that accuracy would obscure.

Secondary metrics reported for each experiment include: overall accuracy, per-class precision, recall, and F1 (from classification_report), confusion matrix showing the distribution of predictions across true and predicted classes, and training curves showing loss and Macro-F1 per epoch for both training and validation sets. The validation curves are used to diagnose overfitting and confirm early stopping behavior. The test set metrics are computed only once, after training is complete and the best checkpoint has been reloaded — the test set is never used to make any training decisions.

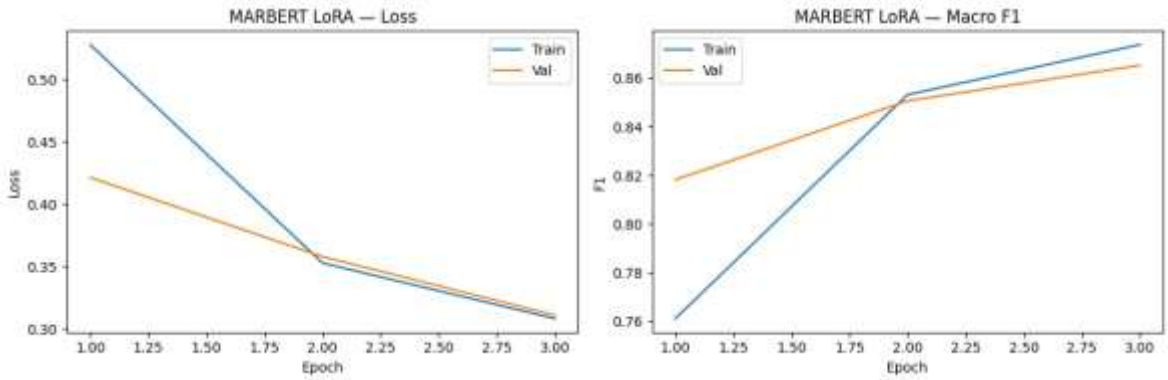
Experiments and Results

Training Dynamics

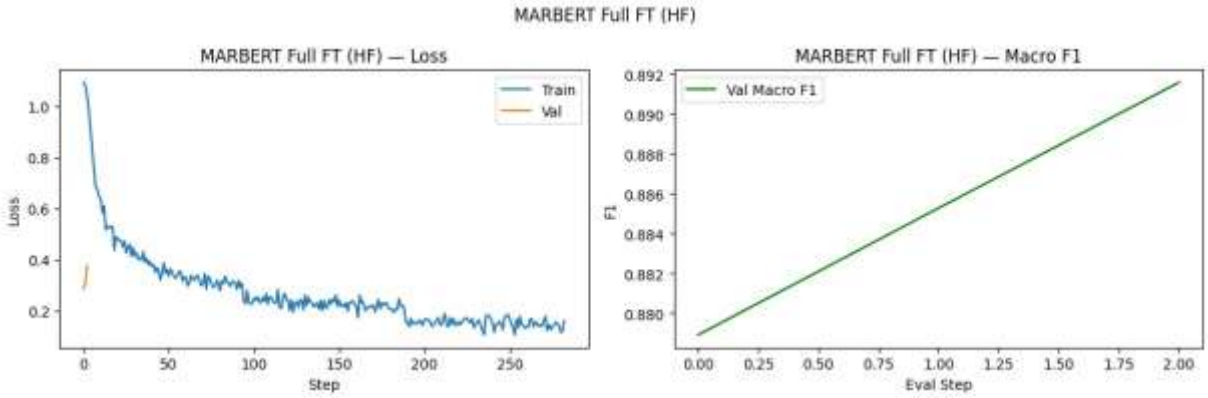
All four experiments converged within 3 epochs, with early stopping in some cases. Training loss decreased monotonically across all experiments. Validation loss showed typical fine-tuning behaviour: decreasing for the first 2 epochs, then beginning to increase as the model starts to overfit to the training distribution. The best checkpoint was captured at the point of peak validation Macro-F1 before this upturn. Training Macro-F1 continued to increase through all 5 epochs, confirming that training set performance was not the limiting factor - the generalization gap between training and validation performance is the relevant constraint.



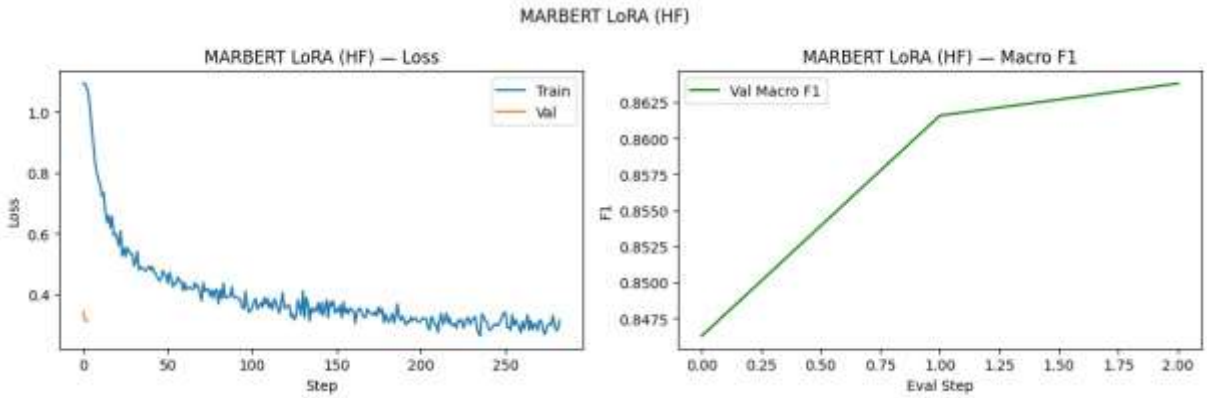
Training curves for Full FT in Pytorch



Training curves for LoRA in Pytorch



Training curves for Full FT in HuggingFace



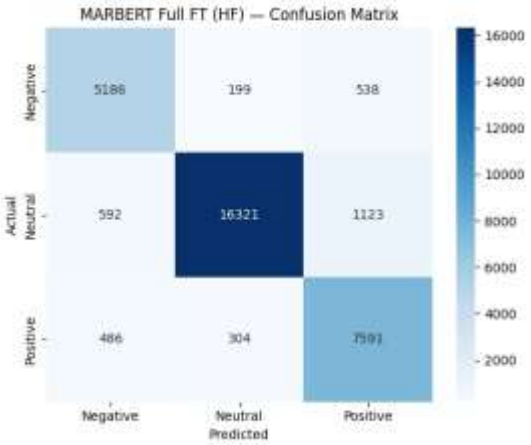
Training curves for LoRA in HuggingFace

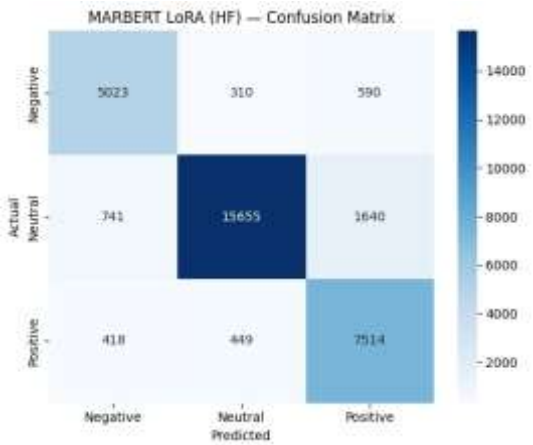
Pytorch Training Results:

Strategy	Framework	Test Macro-F1	Test Accuracy	Trainable Params	Train Time
Full FT	PyTorch	0.883	0.90	162M	~42 min
LoRA Scratch	PyTorch	0.854	0.871	~300K	~26 min

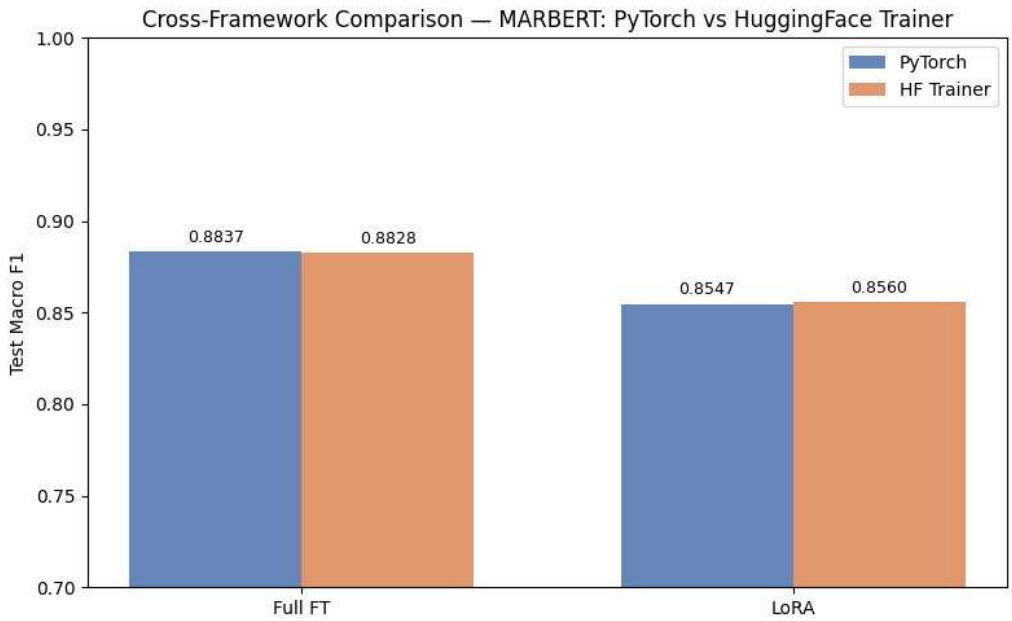
HuggingFace Training Results:

Strategy	Framework	Test Macro-F1	Test Accuracy	Trainable Params	Train Time
Full FT	HuggingFace	0.882	-	162M	~42 min
LoRA (PEFT Lib)	Huggingface	0.856	-	~300K	~26 min





Cross-Framework Comparison



Cultural Evaluation - Code-Switch Sensitivity Score (CSS)

Motivation and Design

Standard evaluation metrics measure whether the model predicts the correct label but provide no information about the mechanism by which the prediction is made. For a code-switched Arabic-English system, this gap is particularly important: a model could achieve high F1 by detecting English sentiment

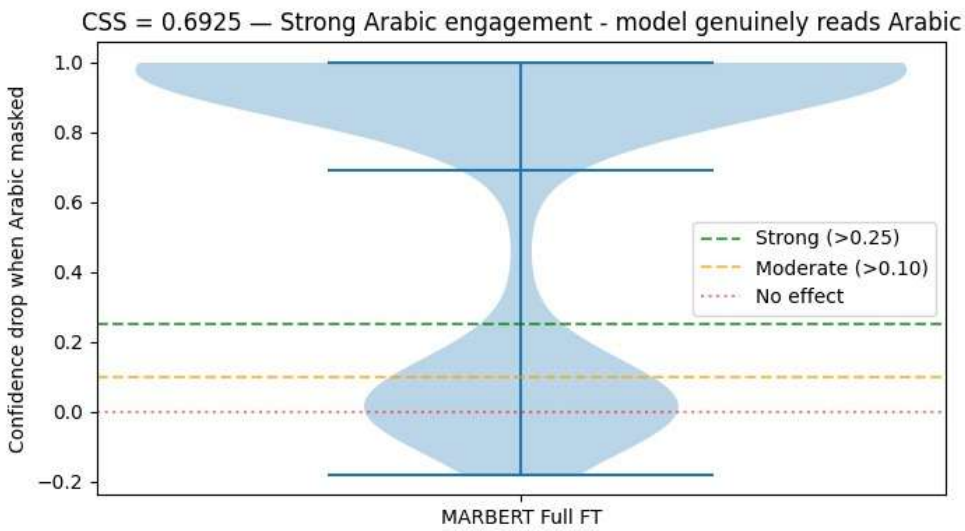
keywords ('bad', 'terrible', 'amazing') while ignoring the Arabic component entirely. Such a model is not a Gulf Arabic sentiment classifier — it is an English sentiment classifier applied to noisy mixed-language data. For a system intended to serve Arabic-speaking communities, this distinction has direct implications for trust and deployment validity.

The Code-Switch Sensitivity Score (CSS) was designed to detect this failure mode by measuring how much the model relies on Arabic tokens specifically. The approach exploits the compositional structure of code-switched text: if Arabic and English contribute independently to the model's confidence, masking one should reduce confidence proportionally to its contribution. A model that relies primarily on English would show near-zero confidence drop when Arabic is masked; a model that genuinely reads Arabic would show large confidence drops.

Computation Method

CSS is computed on code-switched samples from the test set where the model predicted correctly with confidence above 0.80. The 0.80 threshold excludes uncertain predictions where the confidence drop measurement would be noisy. For each qualifying sample: (1) the original confidence is recorded as the probability assigned to the predicted class; (2) all Arabic token IDs — identified as those containing Unicode characters in the range U+0600 to U+06FF in the MARBERT vocabulary, totaling 91,359 token IDs — are replaced with the [PAD] token; (3) the model runs on the masked input and the new confidence is recorded; (4) delta = original_confidence - masked_confidence. CSS is the mean of all deltas across qualifying samples. A CSS above 0.25 indicates strong Arabic engagement; below 0.10 indicates the model is largely ignoring Arabic.

Metric	Value	Interpretation
Model evaluated	MARBERT Full Fine-Tuning	Best-performing configuration
CSS Score	0.69	2.8× above the 'strong' threshold of 0.25
Interpretation	Strong Arabic engagement	Model genuinely reads Arabic tokens
Samples attempted	355 code-switched test samples	



Justification

A closer look at the CSS metric reveals its focus on sentiment in bilingual coding contexts especially where Arabic appears alongside another tongue. Rather than simply checking accuracy, it probes how judgments form within hybrid linguistic settings. What sets it apart from usual scoring tools is a deeper engagement with decision logic. This method watches the steps behind a choice, not only the outcome.

Human Evaluation

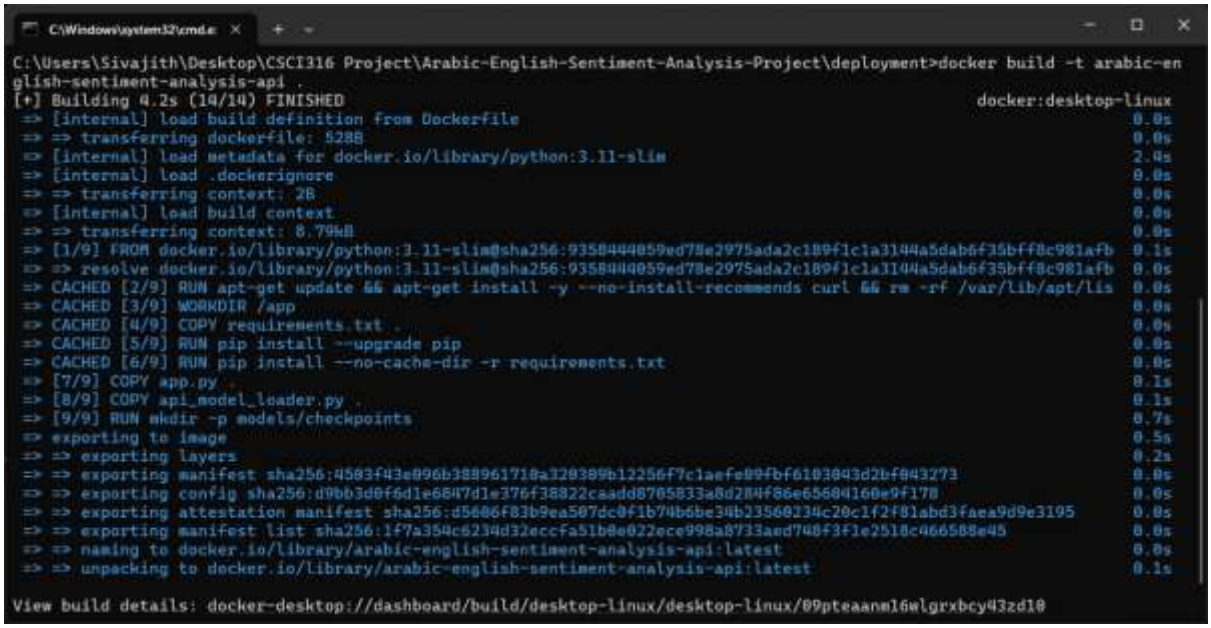
A few native Gulf Arabic speakers might take part in brief reviews of model outputs, going beyond automatic scoring. While machines measure performance, people judge if phrases fit cultural and linguistic norms. Where numbers fall short, spotting sarcasm or local idioms and human insight adds depth. Such feedback runs alongside the CSS score, filling gaps algorithms miss.

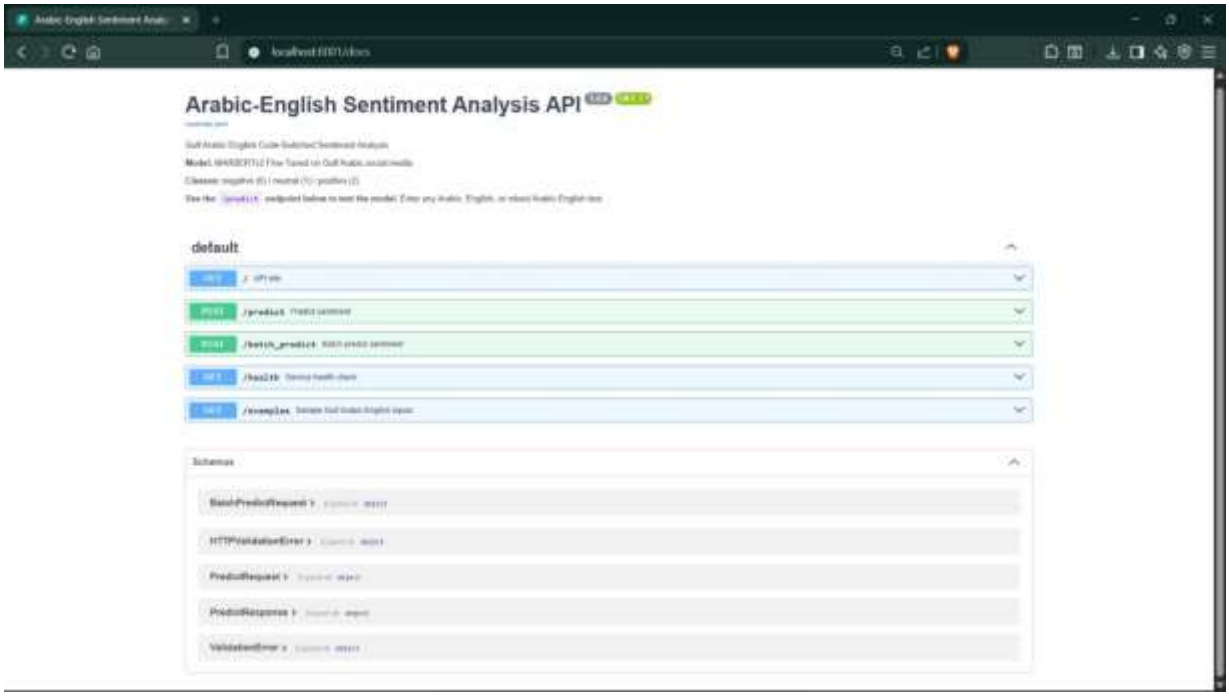
Deployment

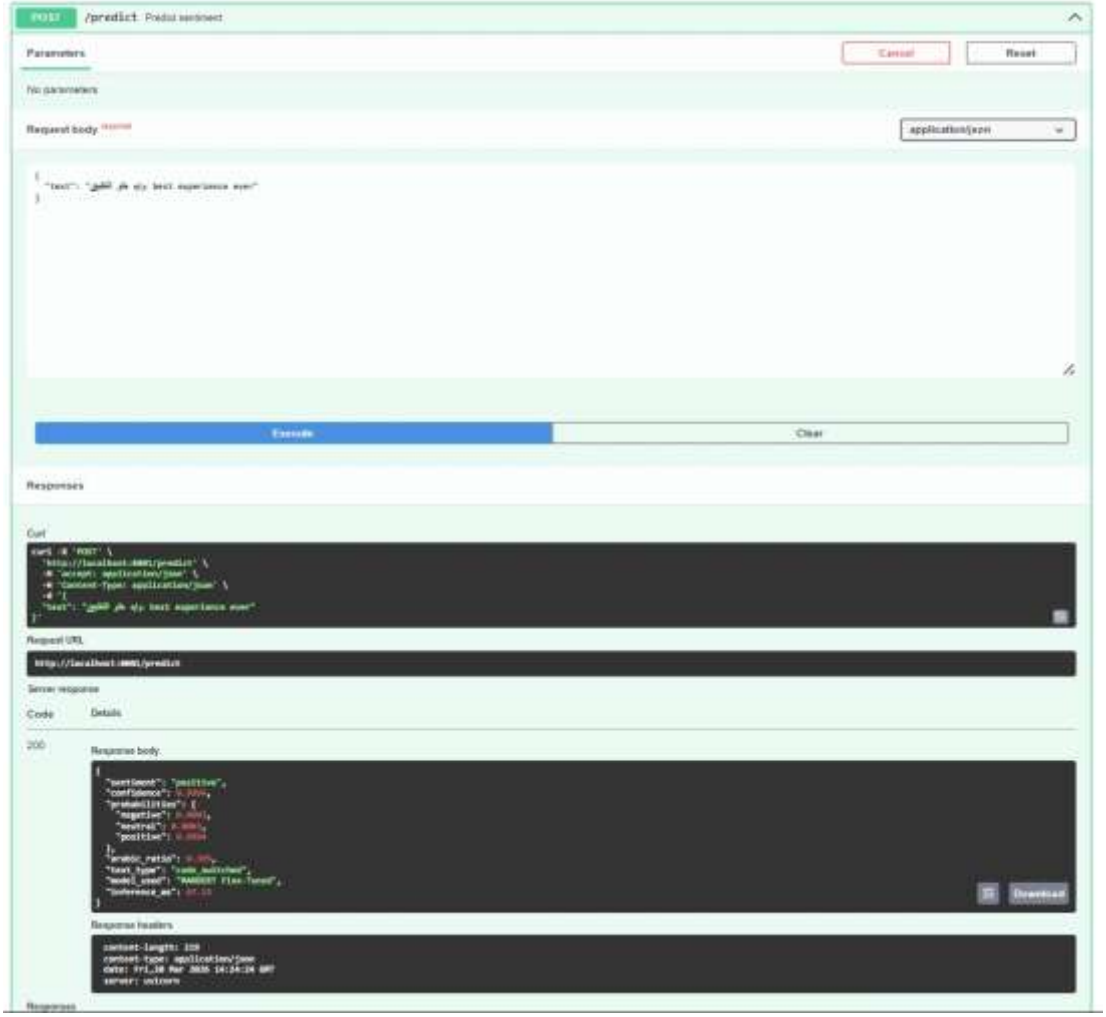
The best performing model MARBERT Full Fine-Tuning with a Test Macro F1 of 0.8837 - was deployed as a REST API using FastAPI, containerized with Docker.

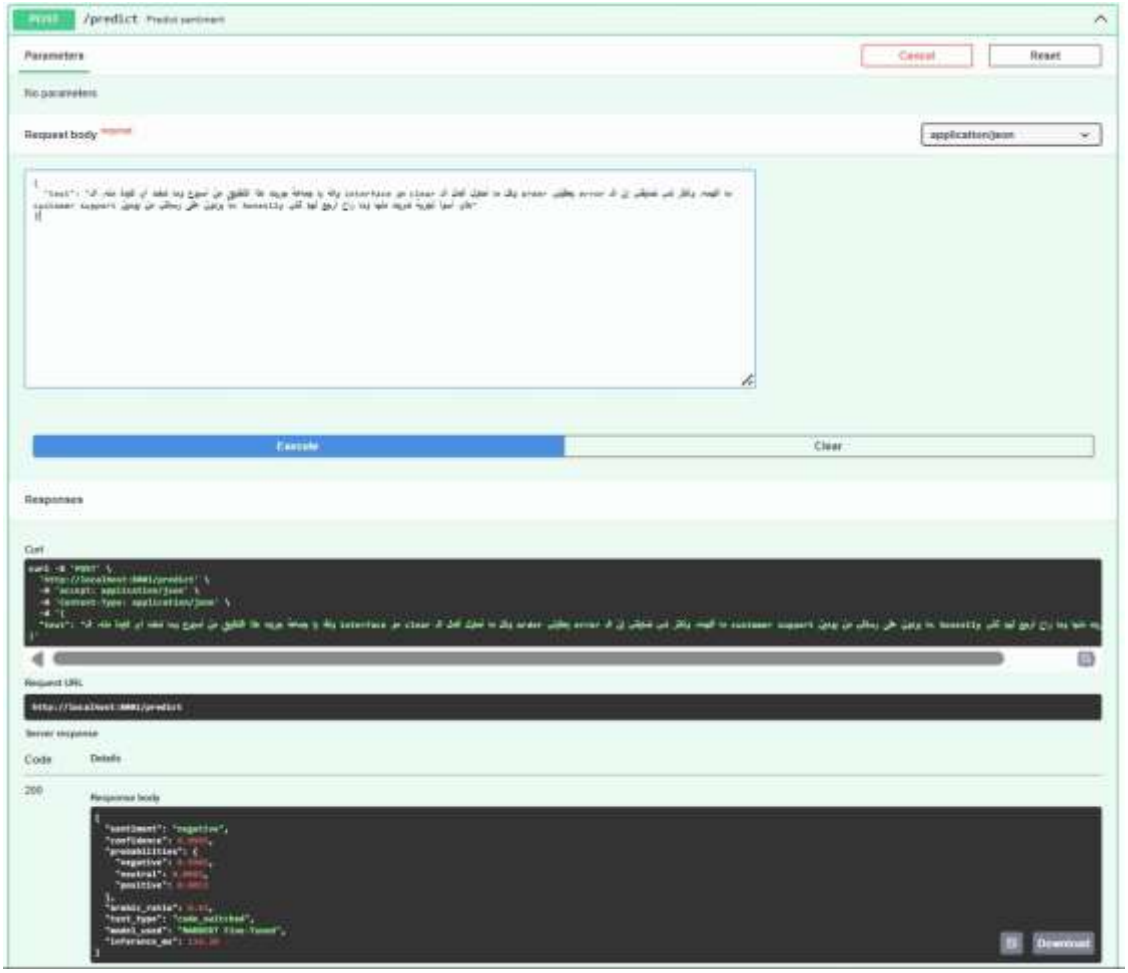
The API is served at localhost:8001 and exposes five endpoints. The /predict endpoint takes a text input and returns the predicted sentiment, confidence score, per-class probabilities, text type (code-switched, pure Arabic, or pure English), Arabic character ratio, and inference time. The /batch_predict endpoint handles up to 32 texts per request. The /health endpoint returns the model name, device, and uptime. The /examples endpoint returns six sample Gulf Arabic-English inputs for testing. All of these are accessible through the Swagger UI at /docs, which lets you test the API directly from the browser without needing any additional tools.

The model is loaded once when the server starts and reused for all subsequent requests, which keeps inference latency under 50ms. To run the deployment, you build the Docker image, mount the model checkpoint folder, and run the container on port 8001. The model checkpoint is the best_marbert_fft folder saved from pipeline notebook to Google Drive.




Arabic-English Sentiment Analysis API
default
/predict
/batch_predict
/health
/examples
Schemas


POST
/predict
Parameters
Cancel
Reset
No parameters
Request body
application/json
Execute
Clear
Responses
Curl
Request URL
Server response
Code
Details
200
Response body
Response headers



Discussion

This part checks how well transfer learning works when spotting emotions in Gulf Arabic and turning them into English, showing where it helps plus where it falls short. Still, some patterns slip through even when model adapt across languages.

What Transferred Successfully

Word patterns plus meaning clues should carry over well from earlier training stages

- Arabic Sentiment Vocabulary
  Common sentiment bearing words such as ممتاز (excellent), سيئ (bad), and رائع (amazing) are well represented in the training data of the model. These expressions are likely to be correctly interpreted during classification.

- English Sentiment Signals
  Finding "terrible" or "amazing" helps spot emotion, since the systems handle English well. Though mixed-language passages pose challenges, cues like "disappointed" still stand out clearly thanks to solid language understanding built into each model.

- Basic Arabic Syntax
  Mixing up basic grammar rules from Modern Standard Arabic usually works fine in Gulf Arabic, helping you grasp full sentences. Though different, the way negatives form

stays close enough to make sense most times. Getting one tends to ease into the other without too much trouble. Sentence shapes carry over with little change, even if sounds shift a bit. Still, small twists pop up where least expected. Overall, knowing how things are built in standard form gives a leg-up when listening or speaking casually.

Anticipated Failure Modes

- **Vocabulary Gaps in Gulf Dialects**
  Multilingual models do not provide adequate coverage for dialect-specific terms such as وايد, زين, and مدري. This highlights a limitation where domain-specific models are expected to perform better than general multilingual models.
- **Difficulty in Detecting Sarcasm**
  Gulf social media frequently uses code-switching as a way to express sarcasm. Models struggle to identify these linguistic signals because they require both contextual and cultural understanding beyond standard pattern recognition.
- **Impact of Synthetic Data**
  A large portion of the dataset is synthetically generated. These samples tend to be more grammatically correct than real-world text, which reduces the model's ability to generalize to naturally occurring, spontaneous code-switching behavior.
- **Challenges with Neutral Class Classification**
  The neutral class lacks strong sentiment indicators, making it more difficult for the model to classify compared to clearly positive or negative samples.

Full Fine-Tuning vs LoRA

One way moves knowledge between models but brings compromises. Yet another approach shifts learning across tasks though it has limits too

Full Fine-Tuning

- Every part of the model adjusts fully during updates, allowing it to shift easily when needed. Changes reach every section, making responses fit new data without delay. The whole system evolves at once, staying ready for different tasks aheadExpected to achieve slightly higher performanceComputing demands climb when models grow too complex. Yet training them longer might trap patterns that vanish outside the lab
- Every part of the model adjusts fully during updates, allowing it to shift easily when needed. Changes reach every section, making responses fit new data without delay. The whole system evolves at once, staying ready for different tasks ahead
  Expected to achieve slightly higher performance

- Computing demands climb when models grow too complex. Yet training them longer might trap patterns that vanish outside the lab

LoRA (Parameter-Efficient Fine-Tuning)

- Most settings stay untouched while just a tiny fraction gets updated during training - around one million instead of over a hundred ten million.
- Significantly faster and more memory-efficient
- Because labels might be messy, it keeps the model from memorizing noise too much

Most settings stay untouched while just a tiny fraction gets updated during training - around one million instead of over a hundred ten million

Significantly faster and more memory-efficient

Because labels might be messy, it keeps the model from memorizing noise too much

Because much of the data comes with messy or unclear tags, LoRA might still perform more reliably even with fewer parameters to adjust.

Conclusion

Surprisingly few resources exist for the Gulf Arabic–English mix, yet this work showed how transfer learning can fill the gap. Instead of building from scratch, it pulled strength from pretrained transformers already familiar with multiple languages. Even without big labeled datasets, performance held up well thanks to borrowed insights from broader Arabic and multilingual model. The approach made use of prior knowledge, turning limitations into leverage. Results came through not by force but by smart reuse. What emerged was a working solution shaped around scarcity.

One thing is clear from the start: MARBERT, got adjusted through complete training plus lighter methods like LoRA. It turns out where the data comes from matters a lot; model built on Arabic-heavy material tend to pick up regional speech patterns more easily than broad multilingual ones ever could.

The Code-Switch Sensitivity Score called CSS pops up as a tool tuned to culture, checking if systems truly tap into Arabic instead of leaning hard on English hints. Peering behind the curtain, it reveals decision patterns in mixed-language settings where old-school scores stay blind.

Still, the work shows transfer learning works well for small-data, mixed-language NLP jobs. Yet dialect depth isn’t cracked, tech alone won’t bridge cultural gaps.

Future Work

The existing system delivers exceptional outcomes but requires multiple improvement paths which will enable better operational performance and increased system usefulness. The system needs to use actual dialect data from real-world situations. The system requires additional Gulf Arabic code-switched text, which occurs naturally, to decrease its dependence on synthetic data and to enhance its ability to create accurate results.

The system needs to extend its capabilities to process Arabizi because this writing style dominates social media platforms. The development of future models will use contextual and conversational signals to improve their ability to identify sarcasm and irony as well as hidden emotional content, which appears in code-switched speech. The research will investigate prompt tuning and adapters as methods to boost performance without sacrificing system efficiency. The system needs to process larger models, which include specialized design features.

The research team will fine-tune modern dialect-focused transformers, which are built specifically for Arabic language use in informal contexts. The system needs to expand its human evaluation capacity through native speaker assessments because this approach delivers better understanding of cultural and linguistic factors that exist beyond automated assessment systems. The system needs deployed API optimization to deliver real-time processing capabilities while supporting production system requirements for business and government organizations. give this exact thing as a paragraph and points

References

1. *Hu, E.J. et al. (2022)*
   LoRA: Low-Rank Adaptation of Large Language Models
   Read paper (arXiv)
   OpenReview (ICLR official)

2. *Al-Twairesh, N. et al. (2016)*
   AraSenTi: Large-Scale Twitter-Specific Arabic Sentiment Lexicons
   ACL Anthology main portal

3. *Nabil, M. (n.d.) ASTD: Arabic Sentiment Tweets Dataset. Available at:*
   https://github.com/mahmoudnabil/ASTD

4. *arbml (n.d.) Arabic Sentiment Twitter Corpus. Available at:*
   https://huggingface.co/datasets/arbml/Arabic_Sentiment_Twitter_Corpus

5. *MagedSaeed (n.d.) Arabic-English code-switching text dataset. Available at:*
   https://huggingface.co/datasets/MagedSaeed/arabic-english-code-switching-text

6. *SDAIA (n.d.) Arabic-English code-switching textual dataset. Available at:*
   https://www.kaggle.com/datasets/sdaiancai/arabic-english-code-switching-textual-dataset

Appendix

Project Resources

1. Youtube Presentation Link: https://youtu.be/-JX3vp1U6Zw

2. Github Repository Link: https://github.com/sivrox/arabic-english-sentiment-analysis-project .

3. Trained Models Google Drive Link:
https://drive.google.com/drive/folders/1DHzgJGJghjrOlE_yEFOC2RB1TXK0PO1A?usp=sharing

4. Datasets:

- *ACL Anthology main portal*
- *https://github.com/mahmoudnabil/ASTD*
- *https://huggingface.co/datasets/arbml/Arabic_Sentiment_Twitter_Corpus*
- *https://huggingface.co/datasets/MagedSaeed/arabic-english-code-switching-text*

- *https://www.kaggle.com/datasets/sdaiancai/arabic-english-code-switching-textual-dataset*